

Interacting with LLM

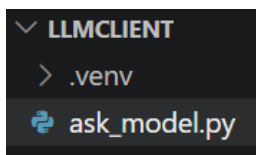
In this module we start experimenting with how to interact with an LLM from a programming language like python. In the process, we explain a few important concepts and terms that you should know and understand.

1. Create a project folder named LLMClient.
2. Open LLMClient with VSCode (right-click the folder and click menu **Open with Code**).
3. Create a python virtual environment. You already did this in the previous module. Go there and check the steps if you forgot how to.
4. In VSCode, while LLMCLIENT is selected, create a file named `ask_model.py`.
5. If the Terminal pane is not visible, click menu View → Terminal to bring it into view. We want to use the `lmstudio` python package. Because `lmstudio` is an external package, we first must install it into the project. From the TERMINAL command prompt in VSCode, type the following command and press the Enter key:

```
pip install lmstudio
```

(wait for installation to complete)

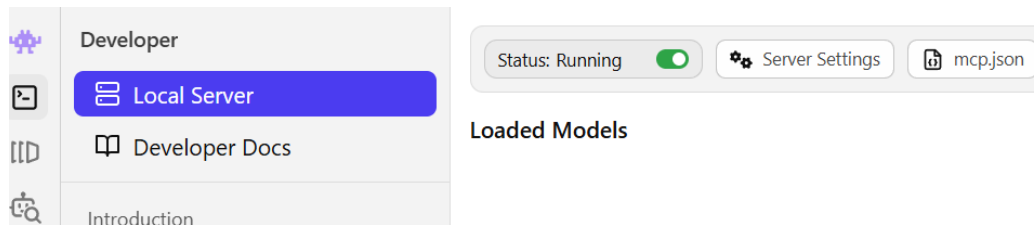
So far, your project structure should look like this:



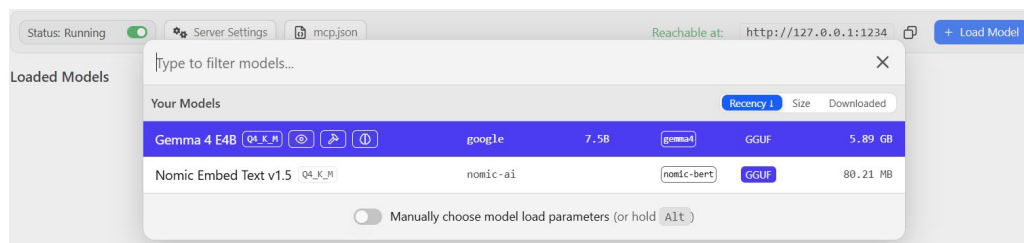
6. Open `ask_model.py` and type this code:

```
import lmstudio as lms
```

7. The first thing to do is get a handle to the model you want to use. To do that we need the model identifier (sometimes called model key). You get the model identifier from the **Developer** → **Local server** page in LM Studio. If you go there and see nothing under **Loaded Models**, then you must first load a model:

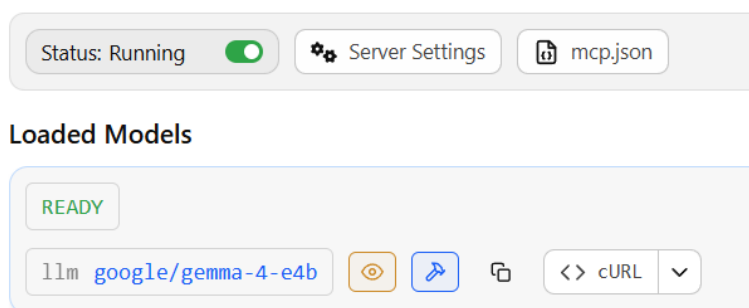


Click the **+ Load Model** button and choose the model to load (in my case I want to load the **Gemma 4 E4B** model from Google).



(wait for the model to load; may take up to a minute).

If the model successfully loads, you should now see it displayed under the **Loaded Models** label. If the server status shows that it is not running, toggle the button to make it run (Status should switch to Running):



Now that we have the model loaded and running, we need to get a handle to it. You can think of the handle as an object that allows us to use the model. Add this code:

```
model = lms.llm("google/gemma-4-e4b")
```

NOTE: In the above code I am using a model with model identifier `google/gemma-4-e4b`. You get this value from the **Model Properties** pane in LM Studio. If you want to use a model other than Google gemma 4, then replace the above string with the identifier of the model you want to use.

At a minimum, the `llm()` function takes a string that represents the model identifier and returns an object of type LLM. In the above code, we store a reference to this object in a variable named `model`, which becomes the handle (or door) to access the functionalities of the LLM.

8. In this program we want to experiment with asking the model a question and getting back a response. Instead of using a variable to store a single question, let's use a list to store a few questions. Then it becomes easier to switch from one question to another. Complete the code to resemble the following:

```
# Import needed packages
import lmstudio as lms

# Get a handle to the model used
model = lms.llm("google/gemma-4-e4b")

# Define a few prompts in a list
prompts = ["What are the largest three cities by population in the USA?",
           "Which US state has the largest economy?",
           "List the names of the countries that are part of the European Union",
           "Briefly outline in less than 10 lines the difference between a bacteria and a virus to a middle school student",
           "I watched a documentary that said that hundreds of millions of years ago, continents on earth were connected " +
           "as one land mass. Is this true?"]
```

9. To the above code, add the following two statements:

```
# Request a response to a prompt (e.g., a question)
result = model.respond(prompts[0])
# Print the returned response
print(result)
```

Here we are passing the first prompt from the list, `prompt[0]`, which is the question about the largest three cities. To pass another prompt, simply change the 0 index value to the index value of the desired prompt from the prompts list.

When done typing the code, click menu **File** → **Save** to save the code in

ask_model.py. Your complete code should now look like this:

```
# Import needed packages
import lmstudio as lms

# Get a handle to the model used
model = lms.llm("google/gemma-4-e4b")

# Define a few prompts in a list
prompts = ["What are the largest three cities by population in the USA?",
           "Which US state has the largest economy?",
           "List the names of the countries that are part of the European Union",
           "Briefly outline in less than 10 lines the difference between a bacteria and a virus to a middle school student",
           "I watched a documentary that said that hundreds of millions of years ago, continents on earth were connected " +
           "as one land mass. Is this true?"]

# Request a response to a prompt (e.g., a question)
result = model.respond(prompts[0])
# Print the returned response
print(result)
```

10. Click the **Run Python File** button to run the code of ask_model.py.

(wait for a minute to two until the model prints back the result).

Read the result. Does the information make sense based on your knowledge?

11. In the model.respond() function, change to another prompt by changing 0 to 1. Save the file and run the code by clicking the **Run Python File** button.

Repeat step 11 by trying the remaining 3 prompts (with indices 2, 3, 4).

12. You probably noticed that it takes a minute or so to get back a response to a given prompt. Another option is to stream the results back in pieces. This allows you to read the pieces of the response almost immediately as they get generated by the LLM.

13. In VSCode, while LLMCLIENT is selected, create a file named ask_model12.py.

14. From ask_model.py, copy the following code, then paste it in ask_model12.py.

```
# Import needed packages
import lmstudio as lms

# Get a handle to the model used
model = lms.llm("google/gemma-4-e4b")

# Define a few prompts in a list
prompts = ["What are the largest three cities by population in the USA?",
            "Which US state has the largest economy?",
            "List the names of the countries that are part of the European Union",
            "Briefly outline in less than 10 lines the difference between a bacteria and a virus to a middle school student",
            "I watched a documentary that said that hundreds of millions of years ago, continents on earth were connected " +
            "as one land mass. Is this true?"]
```

15. Then add the following code to `ask_model12.py`:

```
for fragment in model.respond_stream(prompts[0]):
    print(fragment.content, end="", flush=True)
print() # Advance to a new line at the end of the response
```

The content of `ask_model12.py` should now look like this:

```
# Import needed packages
import lmstudio as lms

# Get a handle to the model used
model = lms.llm("google/gemma-4-e4b")

# Define a few prompts in a list
prompts = ["What are the largest three cities by population in the USA?",
            "Which US state has the largest economy?",
            "List the names of the countries that are part of the European Union",
            "Briefly outline in less than 10 lines the difference between a bacteria and a virus to a middle school student",
            "I watched a documentary that said that hundreds of millions of years ago, continents on earth were connected " +
            "as one land mass. Is this true?"]

for fragment in model.respond_stream(prompts[0]):
    print(fragment.content, end="", flush=True)
print() # Advance to a new line at the end of the response
```

16. Save the content of the file by choosing menu **File → Save all**.

Notice that in this case we are using `respond_stream()` instead of `respond()`. The difference between the two is that `respond()` returns the result after it has assembled the entire response. `respond_stream()`, on the other hand, sends back incremental events (chunks) as the model generates output, rather than waiting for the full completion. This is useful for real-time applications like chat UIs or live transcription.

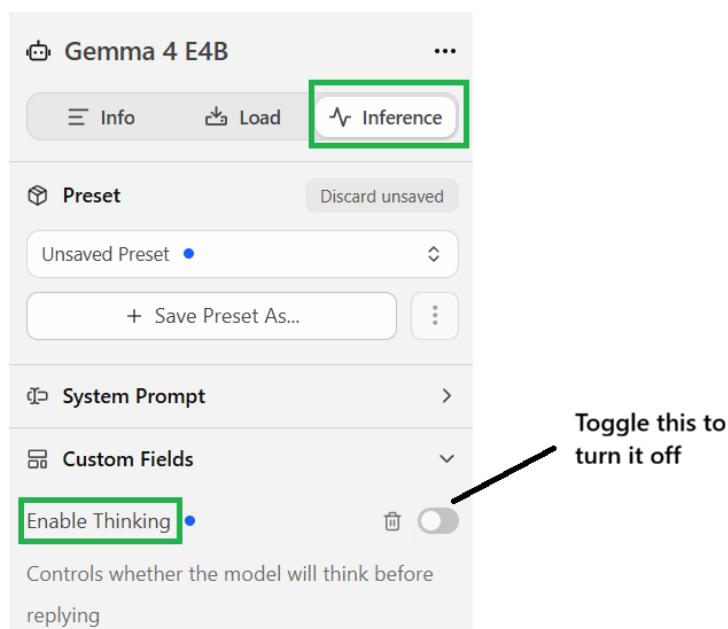
Click the **Run Python File** button to run the code of `ask_model12.py`.

Do you see the difference?

Try another prompt by changing `prompt[0]` to use another index (1 to 4).

17. Another LLM behavior that you might have noticed is that the LLM returns not only the response but its thinking process too: the steps it is outlining for itself to follow to generate an answer. In many cases you don't want to see its thinking process; you just want the answer. For example, imagine you create a chatbot to help customers browsing your website. Customers just want to converse with the chatbot to get quick answers. They don't want to see and read the thinking of the chatbot; In fact, outputting the thinking process in such a case clutters the response and makes it harder for the user to quickly read the relevant answer.

You can turn off the thinking process of the model from the **Inference** tab of the model's properties pane. What you want is to turn off the **Enable Thinking** setting:



18. Now test the new setting you just made:

Pass `prompts[2]` to the `respond_stream()` function.

Click menu **File** → **Save**.

Run the code of `ask_model2.py` by clicking the **Run Python File** button.

Observe the response returned: It should be the list of countries in the European Union without the thinking process text.

Creating a chat conversation with the model

19. We will now write some code to chat with the LLM (kind of like how you do it with ChatGPT).

In VSCode, while LLMCLIENT is selected, create a new file named `chat_with_model.py`. Then enter the following code in `chat_with_model.py`:

```
# Import needed packages
import lmstudio as lms
from sys import stdout

# Define some colors to make our chat interaction look pretty
GREEN = "\033[0;32m"      # User prompt starts with 'You: ' shown in green
BLUE = "\033[34m"         # Model replies starts with 'Assistant: ' shown in blue
YELLOW = "\033[33m"
DEFAULT_COLOR = "\033[0m" # Reset to default color

# Get a handle to the model used
model = lms.llm("google/gemma-4-e4b")

# Print an introduction sentence with instructions how to quit
print()
print("I am an assistant, and I am happy to help you. Just ask questions and we can converse.")
stdout.write(DEFAULT_COLOR + "When ready to terminate the conversation, just type: " + YELLOW + "Thank you" + DEFAULT_COLOR)
print()

# Note: in Python we don't use the new keyword (like Java and C#) to create an object instance of a class.
# Instead, objects are created by calling the class name like a function.

# Create an instance of class Chat.
chat = lms.Chat("You are a task focused AI assistant")

while True:
    stdout.write(GREEN + "You: " + DEFAULT_COLOR)
    user_input = input()
    if len(user_input) == 0:
        continue

    if user_input.lower() == "thank you":
        break

    # Add the user input to the chat history
    chat.add_user_message(user_input)
    prediction_stream = model.respond_stream(
        chat,
        on_message=chat.append,
    )

    stdout.write(BLUE + "Assistant: " + DEFAULT_COLOR)
    for fragment in prediction_stream:
        print(fragment.content, end="", flush=True)
    print()
```

20. Click menu **File** → **Save all** to save the changes you made.

21. Run the code of `chat_with_model.py` by clicking the **Run Python File** button.

NOTE: you can ask it anything, but I suggest you start with the following conversation as a quick test:

You: *Briefly discuss in less than 15 lines the difference between 4WD, AWD, and 4x4?*

Assistant: Wait for it to give you a response

You: *I plan to buy an SUV soon. Which of the 3 types do you recommend if I plan to drive it on gravel and muddy roads?*

Assistant: Wait for it to give you a response

You: *Do you happen to know what brands have 4x4 vehicles?*

Assistant: Wait for it to give you a response

You: Thank you

22. Re-run `chat_with_model.py` and experiment with another conversation of your own.
(Be clear and concise with your questions)

Working with images

For this section, three images were provided to you for testing:

children-playing-outside.jpg

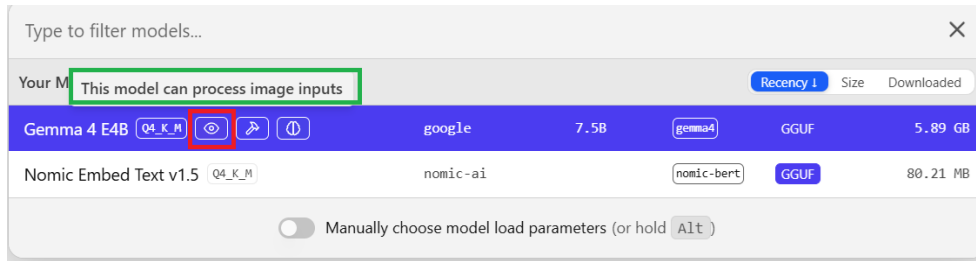
man_with_dog.jpg

man_with_gun.jpg

Download them from the GitHub repo and save them somewhere on your PC.

Some models accept and interpret images. Verify that Gemma 4 E4B is such a model:

- In LM Studio, go to the Developer → Local server page.
- Click the **Load Model** button
- Put the mouse cursor on the vision icon next to Gemma 4 (highlighted in red below):



- You should see a pop-up label telling you that this model accepts images as inputs:

23. In the LMClient VSCode project, make sure LMCLIENT is selected. Click the New File... toolbar button to create a file named `input_image.py`.

24. Enter this code in `input_image.py`:

(In the code below make sure you change the value of variable `image_path` to where you saved `image man_with_dog.jpg` on your PC).

```
# Import needed packages
import lmstudio as lms

# Get a handle to the model used
model = lms.llm("google/gemma-4-e4b")

# Replace path below with the image path on your computer
image_path = "C:/Temp/man_with_dog.jpg"
image_handle = lms.prepare_image(image_path)

# Create an instance of class Chat.
chat = lms.Chat()
chat.add_user_message("Describe this image", images=[image_handle])
result = model.respond(chat)
print(result)
```

25. Save the changes to the file (click menu **File** → **Save**), then run the code by clicking button **Run Python File**.

(Wait for result to get printed; may take up to a minute).

Does the model accurately describe the image content?

26. Change the value of `image_path` to the second image that was given to you (children-playing-outside.jpg). Re-run the code. Does it give you accurate description of the image.

27. Change the value of `image_path` to the third image that was given to you (man_with_gun.jpg). Re-run the code. Does it give you accurate description of the image.

Exercises to do:

Exercise 1

In this exercise, you will make a modified version of `chat_with_model.py` to make it behave slightly differently and be more user-friendly.

We wrote `chat_with_model.py` above in such a way that to start a second chat (say about a different topic), we have to type Thank you, which terminates the program. Then we would have to run the program again to start a new conversation with a new topic. This is not user-friendly. It would be nice if the user could initiate a new chat without having to terminate the program.

- Create a new file and name it `chat_with_model2.py`.
- Copy the code from `chat_with_model.py` and paste it into `chat_with_model2.py`.
- Modify `chat_with_model2.py` to make it start a new chat if the user instructs it to.

HINTS:

- You need to add a new instruction to the user (like the Thank you one) where when they type it, you start a new chat.
- To start a new chat, simply recreate the chat object:

```
# Create an instance of class Chat.  
chat = lms.Chat("You are a task focused AI assistant")
```

- Organize the code to place the creation of the chat object in a separate function and call it whenever you want to create a new chat.
- Test your program by running `chat_with_model2.py`.

Exercise 2

Assume that you have a camera that takes pictures of people entering the door of your school. This picture is fed to an LLM model such as Gemma 4.

Design a system that checks for weapons. If it finds a person entering the school with a weapon, it automatically locks all doors and sends an alert to all staff in the school. For locking the doors and sending the alerts, you can just print:

“All doors locked. Alert sent to all staff”

To test your program, go to Bing or Google images and get images of people carrying weapons and people not carrying weapons.

NOTE: I noticed that the model fails with small images. When you get images, try to get somewhat big-sized ones